Modern Operating Systems — Chapter 3 Memory Management Exercises and Answers

Andrew S. Tanenbaum and Herbert Bos

Contents

Chapter 3: Memory Management

Source note. Questions were extracted from the Chapter 3 problem section of *Modern Operating Systems*, 4th edition. Answers 1–49 were matched to the Chapter 3 answer section of the provided `MOS_4th_answer.pdf`. Exercises 50–55 had no matching answer in that answer section, so their answers are clearly labeled as prepared technical answers rather than answer-manual text.

Exercise 1

Question

The IBM 360 had a scheme of locking 2-KB blocks by assigning each one a 4-bit key and having the CPU compare the key on every memory reference to the 4-bit key in the PSW. Name two drawbacks of this scheme not mentioned in the text.

Answer

First, special hardware is needed to do the comparisons, and it must be fast, since it is used on every memory reference. Second, with 4-bit keys, only 16 programs can be in memory at once (one of which is the operating system).

Source: Answer manual.

Exercise 2

Question

In Fig. 3-3 the base and limit registers contain the same value, 16,384. Is this just an accident, or are they always the same? It is just an accident, why are they the same in this example?

Answer

It is an accident. The base register is 16,384 because the program happened to be loaded at address 16,384. It could have been loaded anywhere. The limit register is 16,384 because the program contains 16,384 bytes. It could have been any length. That the load address happens to exactly match the program length is pure coincidence.

Source: Answer manual.

Exercise 3

Question

A swapping system eliminates holes by compaction. Assuming a random distribution of many holes and many data segments and a time to read or write a 32-bit memory word of 4 nsec, about how long does it take to compact 4 GB? For simplicity, assume that word 0 is part of a hole and that the highest word in memory contains valid data.

Answer

Almost the entire memory has to be copied, which requires each word to be read and then rewritten at a different location. Reading 4 bytes takes 4 nsec, so reading 1 byte takes 1 nsec and writing it takes another 2 nsec, for a total of 2 nsec per byte compacted. This is a rate of 500,000,000 bytes/sec. To copy 4 GB (2^{32} bytes, which is about 4.295×10^9 bytes), the computer needs 2^{32} /500, 000, 000 sec, which is about 859 msec. This number is slightly pessimistic because if the initial hole at the bottom of memory is k bytes, those k bytes do not need to be copied. However, if there are many holes and many data segments, the holes will be small, so k will be small and the error in the calculation will also be small.

Source: Answer manual.

Exercise 4

Question

Consider a swapping system in which memory consists of the following hole sizes in memory order: 10 MB, 4 MB, 20 MB, 18 MB, 7 MB, 9 MB, 12 MB, and 15 MB. Which hole is taken for successive segment requests of
(a) 12 MB
(b) 10 MB
(c) 9 MB for first fit? Now repeat the question for best fit, worst fit, and next fit.

Answer

First fit takes 20 MB, 10 MB, 18 MB. Best fit takes 12 MB, 10 MB, and 9 MB. Worst fit takes 20 MB, 18 MB, and 15 MB. Next fit takes 20 MB, 18 MB, and 9 MB.

Source: Answer manual.

Exercise 5

Question

What is the difference between a physical address and a virtual address?

Answer

Real memory uses physical addresses. These are the numbers that the memory chips react to on the bus. Virtual addresses are the logical addresses that refer to a process' address space. Thus a machine with a 32-bit word can generate virtual addresses up to 4 GB regardless of whether the machine has more or less memory than 4 GB.

Source: Answer manual.

Exercise 6

Question

For each of the following decimal virtual addresses, compute the virtual page number and offset for a 4-KB page and for an 8 KB page: 20000, 32768, 60000.

Answer

For a 4-KB page size the (page, offset) pairs are (4, 3616), (8, 0), and (14, 2656). For an 8-KB page size they are (2, 3616), (4, 0), and (7, 2656).

Source: Answer manual.

Exercise 7

Question

Using the page table of Fig. 3-9, give the physical address corresponding to each of the following virtual addresses:
(a) 20
(b) 4100
(c) 8300

Answer

(a) 8212.
(b) 4100.
(c) 24684.

Source: Answer manual.

Exercise 8

Question

The Intel 8086 processor did not have an MMU or support virtual memory. Nevertheless, some companies sold systems that contained an unmodified 8086 CPU and did paging. Make an educated guess as to how they did it. (Hint: Think about the logical location of the MMU.)

Answer

They built an MMU and inserted it between the 8086 and the bus. Thus all 8086 physical addresses went into the MMU as virtual addresses. The MMU then mapped them onto physical addresses, which went to the bus.

Source: Answer manual.

Exercise 9

Question

What kind of hardware support is needed for a paged virtual memory to work?

Answer

There needs to be an MMU that can remap virtual pages to physical pages. Also, when a page not currently mapped is referenced, there needs to be a trap to the operating system so it can fetch the page.

Source: Answer manual.

Exercise 10

Question

Copy on write is an interesting idea used on server systems. Does it make any sense on a smartphone?

Answer

If the smartphone supports multiprogramming, which the iPhone, Android, and Windows phones all do, then multiple processes are supported. If a process forks and pages are shared between parent and child, copy on write definitely makes sense. A smartphone is smaller than a server, but logically it is not so different.

Source: Answer manual.

Exercise 11

Question

Consider the following C program: int X[N]; int step = M; /* M is some predefined constant */ for (int i = 0; i < N; i += step) X[i] = X[i] + 1;
(a) If this program is run on a machine with a 4-KB page size and 64-entry TLB, what values of M and N will cause a TLB miss for every execution of the inner loop?
(b) Would your answer in part
(a) be different if the loop were repeated many times? Explain.

Answer

For these sizes
(a) M has to be at least 4096 to ensure a TLB miss for every access to an element of X. Since N affects only how many times X is accessed, any value of N will do.
(b) M should still be at least 4,096 to ensure a TLB miss for every access to an element of X. But now N should be greater than 64K to thrash the TLB, that is, X should exceed 256 KB.

Source: Answer manual.

Exercise 12

Question

The amount of disk space that must be available for page storage is related to the maximum number of processes, n, the number of bytes in the virtual address space, v, and the number of bytes of RAM, r. Give an expression for the worst-case disk-space requirements. How realistic is this amount?

Answer

The total virtual address space for all the processes combined is nv, so this much storage is needed for pages. However, an amount r can be in RAM, so the amount of disk storage required is only nv − r. This amount is far more than is ever needed in practice because

rarely will there be n processes actually running and even more rarely will all of them need the maximum allowed virtual memory.

Source: Answer manual.

Exercise 13

Question

If an instruction takes 1 nsec and a page fault takes an additional n nsec, give a formula for the effective instruction time if page faults occur every k instructions.

Answer

A page fault every k instructions adds an extra overhead of n/k μsec to the average, so the average instruction takes 1 + n/k nsec.

Source: Answer manual.

Exercise 14

Question

A machine has a 32-bit address space and an 8-KB page. The page table is entirely in hardware, with one 32-bit word per entry. When a process starts, the page table is copied to the hardware from memory, at one word every 100 nsec. If each process runs for 100 msec (including the time to load the page table), what fraction of the CPU time is devoted to loading the page tables?

Answer

The page table contains 2^{32} /2^{13} entries, which is 524,288. Loading the page table takes 52 msec. If a process gets 100 msec, this consists of 52 msec for loading the page table and 48 msec for running. Thus 52% of the time is spent loading page tables.

Source: Answer manual.

Exercise 15

Question

Suppose that a machine has 48-bit virtual addresses and 32-bit physical addresses.
(a) If pages are 4 KB, how many entries are in the page table if it has only a single level? Explain.
(b) Suppose this same system has a TLB (Translation Lookaside Buffer) with 32 entries. Furthermore, suppose that a program contains instructions that fit into one page and it sequentially reads long integer elements from an array that spans thousands of pages. How effective will the TLB be for this case?

Answer

Under these circumstances:
(a) We need one entry for each page, or $2^{24} = 16 \times 1024 \times 1024$ entries, since there are $36 = 48 - 12$ bits in the page number field.
(b) Instruction addresses will hit 100% in the TLB. The data pages will have a 100 hit rate until the program has moved onto the next data page. Since a 4-KB page contains 1,024 long integers, there will be one TLB miss and one extra memory access for every 1,024 data references.

Source: Answer manual.

Exercise 16

Question

You are given the following data about a virtual memory system: (a)The TLB can hold 1024 entries and can be accessed in 1 clock cycle (1 nsec).
(b) A page table entry can be found in 100 clock cycles or 100 nsec.
(c) The average page replacement time is 6 msec. If page references are handled by the TLB 99% of the time, and only 0.01% lead to a page fault, what is the effective address-translation time?

Answer

The chance of a hit is 0.99 for the TLB, 0.0099 for the page table, and 0.0001 for a page fault (i.e., only 1 in 10,000 references will cause a page fault). The effective address translation time in nsec is then:

Source: Answer manual.

Exercise 17

Question

Suppose that a machine has 38-bit virtual addresses and 32-bit physical addresses.
(a) What is the main advantage of a multilevel page table over a single-level one?
(b) With a two-level page table, 16-KB pages, and 4-byte entries, how many bits should be allocated for the top-level page table field and how many for the nextlevel page table field? Explain.

Answer

Consider,
(a) A multilevel page table reduces the number of actual pages of the page table that need to be in memory because of its hierarchic structure. In fact, in a program with lots of instruction and data locality, we only need the toplevel page table (one page), one instruction page, and one data page.
(b) Allocate 12 bits for each of the three page fields. The offset field requires 14 bits to address 16 KB. That leaves 24 bits for the page fields. Since each entry is 4 bytes, one page can hold 2 12 page table entries and therefore requires 12 bits to index one page. So allocating 12 bits for each of the page fields will address all 238 bytes.

Source: Answer manual.

Exercise 18

Question

Section 3.3.4 states that the Pentium Pro extended each entry in the page table hierarchy to 64 bits but still could only address only 4 GB of memory. Explain how this statement can be true when page table entries have 64 bits.

Answer

The virtual address was changed from (PT1, PT2, Offset) to (PT1, PT2, PT3, Offset). But the virtual address still used only 32 bits. The bit configuration of a virtual address changed from (10, 10, 12) to (2, 9, 9, 12)

Source: Answer manual.

Exercise 19

Question

A computer with a 32-bit address uses a two-level page table. Virtual addresses are split into a 9-bit top-level page table field, an 11-bit second-level page table field, and an offset. How large are the pages and how many are there in the address space?

Answer

Twenty bits are used for the virtual page numbers, leaving 12 over for the offset. This yields a 4-KB page. Twenty bits for the virtual page implies 2 20 pages.

Source: Answer manual.

Exercise 20

Question

A computer has 32-bit virtual addresses and 4-KB pages. The program and data together fit in the lowest page (0–4095) The stack fits in the highest page. How many entries are needed in the page table if traditional (one-level) paging is used? How many page table entries are needed for two-level paging, with 10 bits in each part?

Answer

For a one-level page table, there are 2 32 /2 12 or 1M pages needed. Thus the page table must have 1M entries. For two-level paging, the main page table has 1K entries, each of which points to a second page table. Only two of these are used. Thus, in total only three page table entries are needed, one in the top-level table and one in each of the lower-level tables.

Source: Answer manual.

Exercise 21

Question

Below is an execution trace of a program fragment for a computer with 512-byte pages. The program is located at address 1020, and its stack pointer is at 8192 (the stack grows toward 0). Give the page reference string generated by this program. Each instruction occupies 4 bytes (1 word) including immediate constants. Both instruction and data references count in the reference string. Load word 6144 into register 0 Push register 0 onto the stack Call a procedure at 5120, stacking the return address Subtract the immediate

constant 16 from the stack pointer Compare the actual parameter to the immediate constant 4 Jump if equal to 5152

Answer

The code and reference string are as follows LOAD 6144,R0 1(I), 12(D) PUSH R0 2(I), 15(D) CALL 5120 2(I), 15(D) JEQ 5152 10(I) The code (I) indicates an instruction reference, whereas (D) indicates a data reference.

Source: Answer manual.

Exercise 22

Question

A computer whose processes have 1024 pages in their address spaces keeps its page tables in memory. The overhead required for reading a word from the page table is 5 nsec. To reduce this overhead, the computer has a TLB, which holds 32 (virtual page, physical page frame) pairs, and can do a lookup in 1 nsec. What hit rate is needed to reduce the mean overhead to 2 nsec?

Answer

The effective instruction time is $1h + 5(1 - h)$, where h is the hit rate. If we equate this formula with 2 and solve for h, we find that h must be at least 0.75.

Source: Answer manual.

Exercise 23

Question

How can the associative memory device needed for a TLB be implemented in hardware, and what are the implications of such a design for expandability?

Answer

An associative memory essentially compares a key to the contents of multiple registers simultaneously. For each register there must be a set of comparators that compare each bit in the register contents to the key being searched for. The number of gates (or transistors) needed to implement such a device is a linear function of the number of registers, so expanding the design gets expensive linearly.

Source: Answer manual.

Exercise 24

Question

A machine has 48-bit virtual addresses and 32-bit physical addresses. Pages are 8 KB. How many entries are needed for a single-level linear page table?

Answer

With 8-KB pages and a 48-bit virtual address space, the number of virtual pages is 248 /2 13 , which is 235 (about 34 billion).

Source: Answer manual.

Exercise 25

Question

A computer with an 8-KB page, a 256-KB main memory, and a 64-GB virtual address space uses an inverted page table to implement its virtual memory. How big should the hash table be to ensure a mean hash chain length of less than 1? Assume that the hashtable size is a power of two.

Answer

The main memory has 228 $/2^{13} = 32{,}768$ pages. A 32K hash table will have a mean chain length of 1. To get under 1, we have to go to the next size, 65,536 entries. Spreading 32,768 entries over 65,536 table slots will give a mean chain length of 0.5, which ensures fast lookup.

Source: Answer manual.

Exercise 26

Question

A student in a compiler design course proposes to the professor a project of writing a compiler that will produce a list of page references that can be used to implement the optimal page replacement algorithm. Is this possible? Why or why not? Is there anything that could be done to improve paging efficiency at run time?

Answer

This is probably not possible except for the unusual and not very useful case of a program whose course of execution is completely predictable at compilation time. If a compiler collects information about the locations in the code of calls to procedures, this information might be used at link time to rearrange the object code so that procedures were located close to the code that calls them. This would make it more likely that a procedure would be on the same page as the calling code. Of course this would not help much for procedures called from many places in the program.

Source: Answer manual.

Exercise 27

Question

Suppose that the virtual page reference stream contains repetitions of long sequences of page references followed occasionally by a random page reference. For example, the sequence: 0, 1, ... , 511, 431, 0, 1, ... , 511, 332, 0, 1, ... consists of repetitions of the sequence 0, 1, ... , 511 followed by a random reference to pages 431 and 332.
(a) Why will the standard replacement algorithms (LRU, FIFO, clock) not be effective in handling this workload for a page allocation that is less than the sequence length?
(b) If this program were allocated 500 page frames, describe a page replacement approach that would perform much better than the LRU, FIFO, or clock algorithms.

Answer

Under these circumstances
(a) Every reference will page fault unless the number of page frames is 512, the length of the entire sequence.
(b) If there are 500 frames, map pages 0–498 to fixed frames and vary only one frame.

Source: Answer manual.

Exercise 28

Question

If FIFO page replacement is used with four page frames and eight pages, how many page faults will occur with the reference string $0172^{32}7103$ if the four frames are initially empty? Now repeat this problem for LRU.

Answer

The page frames for FIFO are as follows: x0172333300 xx017222233 xxx01777722 xxxx0111177 The page frames for LRU are as follows: x0172^{32}7103 xx0172^{32}710 xxx01773271 xxxx0111327 FIFO yields six page faults; LRU yields seven.

Source: Answer manual.

Exercise 29

Question

Consider the page sequence of Fig. 3-15(b). Suppose that the R bits for the pages B through A are 11011011, respectively. Which page will second chance remove?

Answer

The first page with a 0 bit will be chosen, in this case D.

Source: Answer manual.

Exercise 30

Question

A small computer on a smart card has four page frames. At the first clock tick, the R bits are 0111 (page 0 is 0, the rest are 1). At subsequent clock ticks, the values are 1011, 1010, 1101, 0010, 1010, 1100, and 0001. If the aging algorithm is used with an 8-bit counter, give the values of the four counters after the last tick.

Answer

The counters are Page 0: 0110110 Page 1: 01001001 Page 2: 00110111 Page 3: 10001011

Source: Answer manual.

Exercise 31

Question

Give a simple example of a page reference sequence where the first page selected for replacement will be different for the clock and LRU page replacement algorithms. Assume that a process is allocated 3=three frames, and the reference string contains page numbers from the set 0, 1, 2, 3.

Answer

The sequence: 0, 1, 2, 1, 2, 0, 3. In LRU, page 1 will be replaced by page 3. In clock, page 1 will be replaced, since all pages will be marked and the cursor is at page 0.

Source: Answer manual.

Exercise 32

Question

In the WSClock algorithm of Fig. 3-20(c), the hand points to a page with R = 0. If τ = 400, will this page be removed? What about if τ = 1000?

Answer

The age of the page is $2204 - 12^{13} = 991$. If $\tau = 400$, it is definitely out of the working set and was not recently referenced so it will be evicted. The $\tau = 1000$ situation is different. Now the page falls within the working set (barely), so it is not removed.

Source: Answer manual.

Exercise 33

Question

Suppose that the WSClock page replacement algorithm uses a τ of two ticks, and the system state is the following: Page Time stamp V R M 0 6 1 0 1 1 9 1 1 0 2 9 1 1 1 3 7 1 0 0 4 4 0 0 0 where the three flag bits V, R, and M stand for Valid, Referenced, and Modified, respectively.
(a) If a clock interrupt occurs at tick 10, show the contents of the new table entries. Explain. (You can omit entries that are unchanged.)
(b) Suppose that instead of a clock interrupt, a page fault occurs at tick 10 due to a read request to page 4. Show the contents of the new table entries. Explain. (You can omit entries that are unchanged.)

Answer

Consider,
(a) For every R bit that is set, set the time-stamp value to 10 and clear all R bits. You could also change the (0,1) R-M entries to (0,0*). So the entries for pages 1 and 2 will change to: Page Time stamp V R M 0 6 1 0 0* 1 10 1 0 0 2 10 1 0 1
(b) Evict page 3 (R = 0 and M = 0) and load page 4: Page Time stamp V R M Notes 0 6 1 0 1 1 9 1 1 0 2 9 1 1 1 3 7 0 0 0 Changed from 7 (1,0,0) 4 10 1 1 0 Changed from 4 (0,0,0)

Source: Answer manual.

Exercise 34

Question

A student has claimed that "in the abstract, the basic page replacement algorithms (FIFO, LRU, optimal) are identical except for the attribute used for selecting the page to be replaced."
(a) What is that attribute for the FIFO algorithm? LRU algorithm? Optimal algorithm?
(b) Give the generic algorithm for these page replacement algorithms.

Answer

Consider,
(a) The attributes are: (FIFO) load time; (LRU) latest reference time; and (Optimal) nearest reference time in the future.
(b) There is the labeling algorithm and the replacement algorithm. The labeling algorithm labels each page with the attribute given in part a. The replacement algorithm evicts the page with the smallest label.

Source: Answer manual.

Exercise 35

Question

How long does it take to load a 64-KB program from a disk whose average seek time is 5 msec, whose rotation time is 5 msec, and whose tracks hold 1 MB
(a) for a 2-KB page size?
(b) for a 4-KB page size? The pages are spread randomly around the disk and the number of cylinders is so large that the chance of two pages being on the same cylinder is negligible.

Answer

The seek plus rotational latency is 10 msec. For 2-KB pages, the transfer time is about 0.009766 msec, for a total of about 10.009766 msec. Loading 32 of these pages will take about 320.21 msec. For 4-KB pages, the transfer time is doubled to about 0.01953 msec, so the total time per page is 10.01953 msec. Loading 16 of these pages takes about 160.3125 msec. With such fast disks, all that matters is reducing the number of transfers (or putting the pages consecutively on the disk).

Source: Answer manual.

Exercise 36

Question

A computer has four page frames. The time of loading, time of last access, and the R and M bits for each page are as shown below (the times are in clock ticks): Page Loaded Last ref. R M 0 126 280 1 0 1 230 265 0 1 2 140 270 0 0 3 110 285 1 1
(a) Which page will NRU replace?
(b) Which page will FIFO replace?
(c) Which page will LRU replace?
(d) Which page will second chance replace?

Answer

NRU removes page 2. FIFO removes page 3. LRU removes page 1. Second chance removes page 2.

Source: Answer manual.

Exercise 37

Question

Suppose that two processes A and B share a page that is not in memory. If process A faults on the shared page, the page table entry for process A must be updated once the page is read into memory.
(a) Under what conditions should the page table update for process B be delayed even though the handling of process A's page fault will bring the shared page into memory? Explain.
(b) What is the potential cost of delaying the page table update?

Answer

Sharing pages brings up all kinds of complications and options:
(a) The page table update should be delayed for process B if it will never access the shared page or if it accesses it when the page has been swapped out again. Unfortunately, in the general case, we do not know what process B will do in the future.
(b) The cost is that this lazy page fault handling can incur more page faults. The overhead of each page fault plays an important role in determining if this strategy is more efficient. (Aside: This cost is similar to that faced by the copy-on-write strategy for supporting some UNIX fork system call implementations.)

Source: Answer manual.

Exercise 38

Question

Consider the following two-dimensional array: int X[64][64]; Suppose that a system has four page frames and each frame is 128 words (an integer occupies one word). Programs that manipulate the X array fit into exactly one page and always occupy page 0. The data are swapped in and out of the other three frames. The X array is stored in row-major order (i.e., X[0][1] follows X[0][0] in memory). Which of the two code fragments shown below will generate the lowest number of page faults? Explain and compute the total number of page faults. Fragment A for (int j = 0; j < 64; j++) for (int i = 0; i < 64; i++) X[i][j] = 0; Fragment B for (int i = 0; i < 64; i++) for (int j = 0; j < 64; j++) X[i][j] = 0;

Answer

Fragment B since the code has more spatial locality than Fragment A. The inner loop causes only one page fault for every other iteration of the outer loop. (There will be only 32 page faults.) [Aside (Fragment A): Since a frame is 128 words, one row of the X array occupies half of a page (i.e., 64 words). The entire array fits into $64 \times 32/128 = 16$ frames. The inner loop of the code steps through consecutive rows of X for a given column. Thus, every other reference to X[i][j] will cause a page fault. The total number of page faults will be $64 \times 64/2 = 2,048$].

Source: Answer manual.

Exercise 39

Question

You have been hired by a cloud computing company that deploys thousands of servers at each of its data centers. They have recently heard that it would be worthwhile to handle a page fault at server A by reading the page from the RAM memory of some other server rather than its local disk drive.
(a) How could that be done?
(b) Under what conditions would the approach be worthwhile? Be feasible?

Answer

It can certainly be done.
(a) The approach has similarities to using flash memory as a paging device in smartphones except now the virtual swap area is a RAM located on a remote server. All of the software infrastructure for the virtual swap area would have to be developed.
(b) The approach might be worthwhile by noting that the access time of disk drives is in the millisecond range while the access time of RAM via a network connection is in the microsecond range if the software overhead is not too high. But the approach might make

sense only if there is lots of idle RAM in the server farm. And then, there is also the issue of reliability. Since RAM is volatile, the virtual swap area would be lost if the remote server went down.

Source: Answer manual.

Exercise 40

Question

One of the first timesharing machines, the DEC PDP-1, had a (core) memory of 4K 18-bit words. It held one process at a time in its memory. When the scheduler decided to run another process, the process in memory was written to a paging drum, with 4K 18-bit words around the circumference of the drum. The drum could start writing (or reading) at any word, rather than only at word 0. Why do you suppose this drum was chosen?

Answer

The PDP-1 paging drum had the advantage of no rotational latency. This saved half a rotation each time memory was written to the drum.

Source: Answer manual.

Exercise 41

Question

A computer provides each process with 65,536 bytes of address space divided into pages of 4096 bytes each. A particular program has a text size of 32,768 bytes, a data size of 16,386 bytes, and a stack size of 15,870 bytes. Will this program fit in the machine's address space? Suppose that instead of 4096 bytes, the page size were 512 bytes, would it then fit? Each page must contain either text, data, or stack, not a mixture of two or three of them.

Answer

The text is eight pages, the data are five pages, and the stack is four pages. The program does not fit because it needs 17 4096-byte pages. With a 512-byte page, the situation is different. Here the text is 64 pages, the data are 33 pages, and the stack is 31 pages, for a total of 128 512-byte pages, which fits. With the small page size it is OK, but not with the large one.

Source: Answer manual.

Exercise 42

Question

It has been observed that the number of instructions executed between page faults is directly proportional to the number of page frames allocated to a program. If the available memory is doubled, the mean interval between page faults is also doubled. Suppose that a normal instruction takes 1 microsec, but if a page fault occurs, it takes 2001 μsec (i.e., 2 msec) to handle the fault. If a program takes 60 sec to run, during which time it gets 15,000 page faults, how long would it take to run if twice as much memory were available?

Answer

The program is getting 15,000 page faults, each of which uses 2 msec of extra processing time. Together, the page fault overhead is 30 sec. This means that of the 60 sec used, half was spent on page fault overhead, and half on running the program. If we run the program with twice as much memory, we get half as many memory page faults, and only 15 sec of page fault overhead, so the total run time will be 45 sec.

Source: Answer manual.

Exercise 43

Question

A group of operating system designers for the Frugal Computer Company are thinking about ways to reduce the amount of backing store needed in their new operating system. The head guru has just suggested not bothering to save the program text in the swap area at all, but just page it in directly from the binary file whenever it is needed. Under what conditions, if any, does this idea work for the program text? Under what conditions, if any, does it work for the data?

Answer

It works for the program if the program cannot be modified. It works for the data if the data cannot be modified. However, it is common that the program cannot be modified and extremely rare that the data cannot be modified. If the data area on the binary file were overwritten with updated pages, the next time the program was started, it would not have the original data.

Source: Answer manual.

Exercise 44

Question

A machine-language instruction to load a 32-bit word into a register contains the 32-bit address of the word to be loaded. What is the maximum number of page faults this instruction can cause?

Answer

The instruction could lie astride a page boundary, causing two page faults just to fetch the instruction. The word fetched could also span a page boundary, generating two more faults, for a total of four. If words must be aligned in memory, the data word can cause only one fault, but an instruction to load a 32-bit word at address 4094 on a machine with a 4-KB page is legal on some machines (including the x86).

Source: Answer manual.

Exercise 45

Question

Explain the difference between internal fragmentation and external fragmentation. Which one occurs in paging systems? Which one occurs in systems using pure segmentation?

Answer

Internal fragmentation occurs when the last allocation unit is not full. External fragmentation occurs when space is wasted between two allocation units. In a paging system, the wasted space in the last page is lost to internal fragmentation. In a pure segmentation system, some space is invariably lost between the segments. This is due to external fragmentation.

Source: Answer manual.

Exercise 46

Question

When segmentation and paging are both being used, as in MULTICS, first the segment descriptor must be looked up, then the page descriptor. Does the TLB also work this way, with two levels of lookup?

Answer

No. The search key uses both the segment number and the virtual page number, so the exact page can be found in a single match.

Source: Answer manual.

Exercise 47

Question

We consider a program which has the two segments shown below consisting of instructions in segment 0, and read/write data in segment 1. Segment 0 has read/execute protection, and segment 1 has just read/write protection. The memory system is a demandpaged virtual memory system with virtual addresses that have a 4-bit page number, and a 10-bit offset. The page tables and protection are as follows (all numbers in the table are in decimal): Segment 0 Segment 1 Read/Execute Read/Write Virtual Page # Page frame # Virtual Page # Page frame # 0 2 0 On Disk 1 On Disk 1 14 2 11 2 9 3 5 3 6 4 On Disk 4 On Disk 5 On Disk 5 13 6 4 6 8 7 3 7 12 For each of the following cases, either give the real (actual) memory address which results from dynamic address translation or identify the type of fault which occurs (either page or protection fault).
(a) Fetch from segment 1, page 1, offset 3
(b) Store into segment 0, page 0, offset 16
(c) Fetch from segment 1, page 4, offset 28
(d) Jump to location in segment 1, page 3, offset 32

Answer

Here are the results: Address Fault?
(a) (14, 3) No (or 0xD3 or 1110 0011)
(b) NA Protection fault: Write to read/execute segment
(c) NA Page fault
(d) NA Protection fault: Jump to read/write segment

Source: Answer manual.

Exercise 48

Question

Can you think of any situations where supporting virtual memory would be a bad idea, and what would be gained by not having to support virtual memory? Explain.

Answer

General virtual memory support is not needed when the memory requirements of all applications are well known and controlled. Some examples are smart cards, special-purpose processors (e.g., network processors), and embedded processors. In these situations, we should always consider the possibility of using more real memory. If the operating system did not have to support virtual memory, the code would be much simpler and smaller. On the other hand, some ideas from virtual memory may still be profitably exploited, although with different design requirements. For example, program/thread isolation might be paging to flash memory.

Source: Answer manual.

Exercise 49

Question

Virtual memory provides a mechanism for isolating one process from another. What memory management difficulties would be involved in allowing two operating systems to run concurrently? How might these difficulties be addressed?

Answer

This question addresses one aspect of virtual machine support. Recent attempts include Denali, Xen, and VMware. The fundamental hurdle is how to achieve near-native performance, that is, as if the executing operating system had memory to itself. The problem is how to quickly switch to another operating system and therefore how to deal with the TLB. Typically, you want to give some number of TLB entries to each kernel and ensure that each kernel operates within its proper virtual memory context. But sometimes the hardware (e.g., some Intel architectures) wants to handle TLB misses without knowledge of what you are trying to do. So, you need to either handle the TLB miss in software or provide hardware support for tagging TLB entries with a context ID.

Source: Answer manual.

Exercise 50

Question

Plot a histogram and calculate the mean and median of the sizes of executable binary files on a computer to which you have access. On a Windows system, look at all .exe and .dll files; on a UNIX system look at all executable files in /bin, /usr/bin, and /local/bin that are not scripts (or use the file utility to find all executables). Determine the optimal page size for this computer just considering the code (not data). Consider internal fragmentation and page table size, making some reasonable assumption about the size of a page table entry.

Assume that all programs are equally likely to be run and thus should be weighted equally.

Answer

One safe way to do the experiment is to collect executable file sizes, plot them, and then evaluate candidate page sizes by balancing internal fragmentation against page-table memory. The experiment should be run read-only; it does not need administrator privileges. Unix commands:

```
find /bin /usr/bin /usr/local/bin -type f -perm -111 -exec file {} \; |
  grep 'ELF' | cut -d:  -f1 > executables.txt
python3 page_size_study.py executables.txt
```

Windows PowerShell commands:

```
Get-ChildItem C:\Windows\System32 -Include *.exe,*.dll -File -Recurse |
  Select-Object -ExpandProperty FullName > executables.txt
python page_size_study.py executables.txt
```

Example analysis script:

```
import os, statistics, sys
import matplotlib.pyplot as plt

paths = [p.strip() for p in open(sys.argv[1], encoding="utf-8",
errors="ignore")]
sizes = [os.path.getsize(p) for p in paths if os.path.exists(p)]
print("count", len(sizes))
print("mean", statistics.mean(sizes))
print("median", statistics.median(sizes))

pte = 8
for page in [512,1024,2048,4096,8192,16384,32768,65536]:
    frag = sum((page - (s % page)) % page for s in sizes) / len(sizes)
    pt = sum(((s + page - 1) // page) * pte for s in sizes) / len(sizes)
    print(page, "avg_internal_frag", frag, "avg_page_table_bytes", pt,
          "combined", frag + pt)

plt.hist(sizes, bins=50)
plt.xlabel("executable size in bytes")
plt.ylabel("count")
plt.savefig("executable_size_histogram.png")
```

Expected behavior: very small pages reduce internal fragmentation but increase the number of page-table entries; very large pages shrink page tables but waste more space in the last page of each executable. The optimal page size under this simplified model is the candidate with the smallest average value of internal fragmentation plus page-table bytes. Safety notes: scan only local system directories you are allowed to read, do not execute the files, and expect permission-denied errors or very long scans on whole-disk searches.

Source: Independently prepared technical answer.

Exercise 51

Question

Write a program that simulates a paging system using the aging algorithm. The number of page frames is a parameter. The sequence of page references should be read from a file. For a given input file, plot the number of page faults per 1000 memory references as a function of the number of page frames available.

Answer

The aging algorithm keeps an age counter for each resident page. On each reference interval, the counter is shifted right and the current referenced bit is inserted at the high bit. On a page fault, the page with the smallest counter is replaced.

```
from collections import defaultdict
import sys

def simulate(refs, frames, bits=8):
    ages, resident, faults = {}, set(), 0
    for r in refs:
        for p in list(resident):
            ages[p] >>= 1
        if r in resident:
            ages[r] |= 1 << (bits - 1)
            continue
        faults += 1
        if len(resident) >= frames:
            victim = min(resident, key=lambda p:  ages[p])
            resident.remove(victim)
            del ages[victim]
        resident.add(r)
        ages[r] = 1 << (bits - 1)
```

```
        return faults

refs = [int(x) for x in open(sys.argv[1]).read().split()]
for frames in range(1, int(sys.argv[2]) + 1):
    f = simulate(refs, frames)
    print(frames, 1000 * f / len(refs))
```

Run it as:

```
python aging.py references.txt 64 > aging.dat
```

Expected behavior: page faults per 1000 references should generally fall as the number of frames increases, although particular reference strings can have small irregularities. Safety notes: this is a simulator, not a kernel experiment; use modest input files first, validate against a short hand-computed reference string, and reject noninteger input before using large traces.

Source: Independently prepared technical answer.

Exercise 52

Question

Write a program that simulates a toy paging system that uses the WSClock algorithm. The system is a toy in that we will assume there are no write references (not very realistic), and process termination and creation are ignored (eternal life). The inputs will be:

- The reclamation age threshhold
- The clock interrupt interval expressed as number of memory references
- A file containing the sequence of page references

(a) Describe the basic data structures and algorithms in your implementation.
(b) Show that your simulation behaves as expected for a simple (but nontrivial) input example.
(c) Plot the number of page faults and working set size per 1000 memory references.
(d) Explain what is needed to extend the program to handle a page reference stream that also includes writes.

Answer

A toy WSClock simulator can store one frame record per resident page: page number, reference bit R, last-use time, and a next pointer represented by the clock-hand index. Since the exercise excludes writes, there is no dirty bit or write-back scheduling.

```
def wsclock(refs, frames, tau, interrupt):
    clock, hand, time, faults, ws = [], 0, 0, 0, []
    pos = {}
```

```
        for page in refs:
            time += 1
            if page in pos:
                clock[pos[page]]["R"] = 1
            else:
                faults += 1
                if len(clock) < frames:
                    pos[page] = len(clock)
                    clock.append({"page":  page, "R":  1, "last":  time})
                else:
                    while True:
                        f = clock[hand]
                        if f["R"]:
                            f["R"] = 0
                            f["last"] = time
                        elif time - f["last"] > tau:
                            del pos[f["page"]]
                            f.update({"page":  page, "R":  1, "last":  time})
                            pos[page] = hand
                            hand = (hand + 1) % frames
                            break
                        hand = (hand + 1) % frames
            if time % interrupt == 0:
                for f in clock:
                    f["R"] = 0
            ws.append(sum(1 for f in clock if time - f["last"] <= tau))
        return faults, ws
```

Basic algorithm: on a hit, set R. On a fault, scan from the clock hand. If R is set, clear it and refresh the last-use time. If R is clear and the page is older than the threshold, replace it. Otherwise continue scanning. For a simple nontrivial test such as references `1 2 3 1 2 4 1 2 5` with three frames, the first three references fault, later references to 1 and 2 hit, and references to 4 and 5 force replacements after the clock scan finds old pages. To extend the simulator for writes, each record needs a dirty bit; clean old pages can be replaced immediately, while dirty old pages must first be scheduled for write-back and skipped until clean. Safety notes: cap the reference-stream size or stream it from disk for large traces, and avoid treating this simulator as evidence about an actual OS until compared with controlled traces.

Source: Independently prepared technical answer.

Exercise 53

Question

Write a program that demonstrates the effect of TLB misses on the effective memory access time by measuring the per-access time it takes to stride through a large array.
(a) Explain the main concepts behind the program, and describe what you expect the output to show for some practical virtual memory architecture.
(b) Run the program on some computer and explain how well the data fit your expectations.
(c) Repeat part
(b) but for an older computer with a different architecture and explain any major differences in the output.

Answer

This experiment measures how access time changes as the stride crosses cache and TLB reach boundaries. The program touches one integer per stride over a large array and reports nanoseconds per access. I did not run the experiment here.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

static long ns(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1000000000L + ts.tv_nsec;
}

int main(int argc, char **argv) {
    size_t mb = argc > 1 ?  atol(argv[1]) :  512;
    size_t n = (mb * 1024 * 1024) / sizeof(int);
    int *a = calloc(n, sizeof(int));
    volatile long sum = 0;
    if (!a) return 1;
    for (size_t stride = 16; stride <= 65536; stride *= 2) {
        long start = ns();
        size_t accesses = 0;
        for (int rep = 0; rep < 20; rep++)
            for (size_t i = 0; i < n; i += stride / sizeof(int)) {
                sum += a[i];
                accesses++;
            }
        long end = ns();
```

```
            printf("%zu %g\n", stride, (double)(end - start) / accesses);
        }
        free(a);
        return (int)sum;
    }
```

Compile and run on Unix-like systems:

```
cc -O2 tlb_stride.c -o tlb_stride
./tlb_stride 512 > tlb.dat
```

Expected behavior: small strides are dominated by cache behavior; as the stride approaches or exceeds the page size, the program touches many pages with little reuse, so TLB misses raise the average access time. Safety notes: choose an array size comfortably below physical RAM to avoid paging the whole machine; close other heavy programs; repeat runs because CPU frequency scaling and background activity add noise. On older architectures, different page sizes, TLB sizes, cache hierarchies, and replacement policies may move or blur the breakpoints.

Source: Independently prepared technical answer.

Exercise 54

Question

Write a program that will demonstrate the difference between using a local page replacement policy and a global one for the simple case of two processes. You will need a routine that can generate a page reference string based on a statistical model. This model has N states numbered from 0 to N − 1 representing each of the possible page references and a probability p i associated with each state i representing the chance that the next reference is to the same page. Otherwise, the next page reference will be one of the other pages with equal probability.
(a) Demonstrate that the page reference string-generation routine behaves properly for some small N .
(b) Compute the page fault rate for a small example in which there is one process and a fixed number of page frames. Explain why the behavior is correct.
(c) Repeat part
(b) with two processes with independent page reference sequences and twice as many page frames as in part (b).
(d) Repeat part
(c) but using a global policy instead of a local one. Also, contrast the per-process page fault rate with that of the local policy approach.

Answer

Use a reference generator whose next page is the same as the current page with probability `p[i]`; otherwise choose uniformly among the other pages. Then compare local replacement, where each process owns a fixed frame set, with global replacement, where both processes draw from one shared frame pool.

```
import random

def gen(n, probs, length, start=0):
    cur, out = start, []
    for _ in range(length):
        out.append(cur)
        if random.random() >= probs[cur]:
            choices = [x for x in range(n) if x != cur]
            cur = random.choice(choices)
    return out

def lru_faults(refs, frames):
    mem, faults = [], 0
    for p in refs:
        if p in mem:
            mem.remove(p)
        else:
            faults += 1
            if len(mem) == frames:
                mem.pop(0)
        mem.append(p)
    return faults

def global_lru(stream, frames):
    mem, faults = [], [0, 0]
    for pid, page in stream:
        key = (pid, page)
        if key in mem:
            mem.remove(key)
        else:
            faults[pid] += 1
            if len(mem) == frames:
                mem.pop(0)
        mem.append(key)
    return faults
```

For part (a), generate a short stream with small N and count transitions; pages with larger `p[i]` should repeat more often. For part (b), one process with a fixed frame count should show fewer faults as frames increase. For part (c), two independent processes with local allocation and twice the frames should behave roughly like two copies of part (b). For part (d), global replacement may improve total faults when one process needs fewer frames, but it can also let one process steal frames and raise the other process's fault rate. Safety notes: seed the random generator for reproducibility, run long enough streams before trusting rates, and report per-process fault rates rather than only totals.

Source: Independently prepared technical answer.

Exercise 55

Question

Write a program that can be used to compare the effectiveness of adding a tag field to TLB entries when control is toggled between two programs. The tag field is used to effectively label each entry with the process id. Note that a nontagged TLB can be simulated by requiring that all TLB entries have the same tag at any one time. The inputs will be:

- The number of TLB entries available
- The clock interrupt interval expressed as number of memory references
- A file containing a sequence of (process, page references) entries
- The cost to update one TLB entry

(a) Describe the basic data structures and algorithms in your implementation. b) Show that your simulation behaves as expected for a simple (but nontrivial) input example. (c) Plot the number of TLB updates per 1000 references.

Answer

A tagged TLB keeps entries from different processes at the same time by matching both process id and virtual page. An untagged TLB can be simulated by flushing the TLB whenever the process id changes.

```
def simulate(refs, entries, switch_interval, update_cost, tagged):
    tlb, updates, time, last_pid = [], 0, 0, None
    for pid, page in refs:
        time += 1
        if not tagged and last_pid is not None and pid != last_pid:
            updates += len(tlb) * update_cost
            tlb.clear()
        key = (pid, page) if tagged else page
        if key in tlb:
            tlb.remove(key)
```

```
            else:
                updates += update_cost
                if len(tlb) == entries:
                    tlb.pop(0)
            tlb.append(key)
            last_pid = pid
        return updates

refs = [(0,1),(0,2),(1,1),(1,2),(0,1),(1,1),(0,2),(1,2)]
print("untagged", simulate(refs, 4, 2, 1, False))
print("tagged", simulate(refs, 4, 2, 1, True))
```

Basic data structures: an ordered list or deque for LRU TLB entries, a pair `(process,page)` for tagged entries, and counters for TLB updates per 1000 references. Expected behavior: with frequent process switches, the untagged TLB pays repeated flush/refill costs; the tagged TLB preserves entries across switches, so updates per 1000 references should be lower when the working sets fit in the tagged TLB. Safety notes: validate on a hand-written trace before plotting, keep the update cost separate from misses so assumptions are visible, and do not infer actual hardware performance without measuring real context-switch and TLB behavior.

Source: Independently prepared technical answer.